

LARS JANSEN

Senior Software Engineer

Assen, Drenthe, Netherlands | +31 97058049397 | larsjansen0521@outlook.com | linkedin.com/in/lars-jansen-0a37b8393

PROFESSIONAL SUMMARY

Senior **Python** backend engineer in **production & visual creation**, delivering greenfield APIs and ML-ready workflows for image/content generation used by enterprise brands. Ships **FastAPI** services with async patterns, strict validation, and consistent errors; models data in **PostgreSQL** and uses **Redis** for caching and rate-safe concurrency. Runs containerized platforms with **Docker** and **Kubernetes** on **AWS/Azure**, adding observability, safe migrations, and incident playbooks; upgrades legacy Python 3.7/3.8 services to **Python 3.10–3.12** via phased rollouts. Strong focus on **CI/CD**, secrets hygiene, and **SLOs** so firefighting becomes prevention.

SKILLS

- **Backend (Python)** — **Python 3.8–3.12**, **FastAPI 0.95–0.110**, Pydantic v1→v2 migration patterns, asyncio, background workers, WebSockets/SSE, REST contracts, OpenAPI, request validation, idempotency, retries/backoff, multi-tenant patterns, feature flags, safe rollouts
- **Data & Caching** — **PostgreSQL 13–16** (schema design, indexing, partitioning, query tuning, migration discipline), **Redis 6–7** (caching, rate limiting, distributed locks, queue primitives), data backfills, auditing scripts, transactional integrity
- **Cloud & Containers** — **Docker 20–25**, **Kubernetes 1.24–1.29** (deployments, probes, HPA, resource limits, PDBs, ingress), **AWS** (EKS/ECS, S3, RDS, IAM, CloudWatch, SQS/SNS), **Azure** (AKS, Key Vault, Storage, Monitor), IaC with Terraform 1.x patterns
- **Observability, Quality & Security** — OpenTelemetry, structured logging, metrics + tracing, SLO dashboards, alert tuning, contract tests, integration tests, load testing, OAuth2/OIDC, **JWT**, RBAC, secrets management, dependency scanning, incident runbooks

WORK HISTORY

Senior Software Engineer RavenTrack

August 2021 to November 2025

- Designed greenfield **FastAPI** backends for visual-production workflows, enforcing strict request validation and consistent error contracts so clients could integrate without guesswork.
- Built an async job orchestration layer for image/content generation pipelines, using retry/backoff and dead-letter handling to prevent long-running tasks from stalling production queues.
- Implemented a **PostgreSQL** schema strategy with safe migrations, online index changes, and backfill scripts, eliminating release-day lock contention during high-traffic publishing windows.
- Added **Redis** caching with stampede protection and TTL discipline for expensive renders and metadata lookups, stabilizing p95 latency during bursty campaign launches.
- Resolved a critical bug where concurrent retries produced duplicate assets by introducing idempotency keys, unique constraints, and replay-safe handlers across API and worker boundaries.
- Fixed a high-impact “missing thumbnails” incident caused by eventual-consistency timing by adding state-machine transitions, durable retries, and reconciliation jobs with audit trails.
- Upgraded legacy services from Python 3.7/3.8 to **Python 3.10–3.12**, addressing async edge cases, dependency pinning, and phased rollout safety via parallel canaries.
- Migrated validation models from Pydantic v1 patterns to v2-compatible structures, preventing subtle serialization breaks and keeping OpenAPI outputs stable for client SDKs.
- Containerized services with **Docker** and deployed on **Kubernetes**, tightening probes, resource requests/limits, and disruption budgets to reduce restart storms under load.
- Built CI/CD pipelines with quality gates (lint, type checks, tests, image scanning), cutting “works on my machine” failures and making releases predictable even in chaos.
- Instrumented services with **OpenTelemetry** tracing and high-cardinality log correlation, shrinking incident triage from hours to minutes during firefighting rotations.
- Implemented secure secrets handling across **AWS** and **Azure** (least-privilege IAM, vault-backed rotation), closing gaps found during internal security reviews.
- Introduced a versioned API strategy for partner integrations, enabling new capabilities without breaking older clients while deprecations were tracked and communicated.
- Optimized slow render-metadata queries by rewriting joins, adding covering indexes, and validating query plans, preventing timeout cascades during peak brand events.
- Partnered with product owners to ship iterative ML-enabled features safely, adding feature flags, rollout playbooks, and rollback paths to protect production reliability.

Software Engineer Infinite Loop

June 2018 to July 2021

- Delivered Python backend features for a scale-up product team, translating ambiguous requirements into **FastAPI** endpoints with clear contracts and testable acceptance criteria.
- Modernized a legacy synchronous API into async handlers where it mattered, preventing thread exhaustion and improving throughput on upload-heavy processing flows.

- Fixed an intermittent “stuck job” issue by adding heartbeat tracking, visibility timeouts, and worker-side recovery logic so retries became safe and observable.
- Implemented **Redis**-backed rate limiting and lightweight locking to stop concurrent edits from corrupting shared content states during multi-user sessions.
- Improved **PostgreSQL** reliability by splitting hot tables, applying targeted indexes, and introducing migration checks to keep deploys safe under continuous delivery.
- Migrated older container builds to slimmer, reproducible images and standardized health checks, reducing cold-start time and surprise runtime dependency failures.
- Built integration tests around external provider webhooks, catching signature-validation and timestamp-skew bugs before they could break production ingest.
- Resolved a painful timezone/locale defect in scheduled publishing by normalizing all timestamps to UTC, enforcing ISO-8601 serialization, and backfilling affected rows.
- Introduced structured logging and trace propagation across services, making cross-system debugging feasible when multiple teams were shipping changes simultaneously.
- Implemented new features behind flags with staged rollouts and monitoring hooks, allowing safe releases even when requirements changed mid-sprint.
- Collaborated closely with senior engineers on code reviews and incident retros, turning recurring failures into engineering standards and runbook updates.

Software Developer Datamart

October 2015 to May 2018

- Built Python services and utilities that transformed raw operational data into consistent datasets, reducing manual reconciliation work for analysts and operations users.
- Designed and tuned **PostgreSQL** queries for reporting workloads, applying indexing and batching strategies to keep large exports stable and repeatable.
- Fixed recurring data-quality bugs caused by partial writes by enforcing transactional boundaries, retries with idempotent upserts, and integrity checks on ingest.
- Created background processing patterns for long-running imports, adding progress tracking and safe restart behavior so failures did not require manual cleanup.
- Improved performance of heavy reports by rewriting slow joins, introducing summary tables, and validating query plans to prevent timeouts during peak usage.
- Migrated older Python runtime assumptions to newer packaging and dependency management practices, reducing deploy failures due to environment drift.
- Implemented basic observability (structured logs, error aggregation, dashboards) so issues could be found proactively instead of discovered by users first.
- Partnered with stakeholders to convert changing report definitions into maintainable logic, keeping output consistent even when business rules evolved.
- Hardened release processes with repeatable build steps and smoke tests, preventing last-minute surprises when shipping fixes to production.

EDUCATION

Bachelor's degree in Computer Science University of Amsterdam

2011 to 2015

- Built strong foundations in software engineering, data structures, and systems design, with coursework that supports designing reliable services and data-driven applications.

PROJECTS

ML Content Generation Orchestrator (Greenfield)

- Designed a **FastAPI** orchestration service coordinating image/content generation steps, including prompt/template versioning, queue-driven execution, and deterministic retries for repeatable outputs.
- Built a **PostgreSQL** audit trail for every generation request (inputs, parameters, outputs, timing, and errors), enabling reproducibility, compliance-friendly reviews, and fast incident forensics.
- Added **Redis** caching and deduplication to avoid re-running identical expensive generations, protecting cost and latency during campaign spikes while keeping results consistent.